Carnegie Mellon University

CARNEGIE INSTITUTE OF TECHNOLOGY

THESIS

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE DEGREE OF Master of Science

TITLE _____

_____

PRESENTED BY _____

ACCEPTED BY THE DEPARTMENT OF

Information Networking Institute

_____ THESIS ADVISOR _____ DATE

_____ ACADEMIC ADVISOR _____ DATE

_____ DEPARTMENT HEAD _____ DATE

APPROVED BY THE COLLEGE COUNCIL

_____ DEAN _____ DATE

Tool-Orchestrated Agentic Reverse Engineering and Malware Analysis

Submitted in partial fulfillment of the requirements for

the degree of

Master of Science

in

Information Security

Matthew Ung

TODO: add prior degree information

Carnegie Mellon University
Pittsburgh, PA

April, 2026

© Matthew Ung, 2026
All Rights Reserved

Acknowledgements

This draft keeps the acknowledgements in scaffold form until the final thesis version is ready for submission.

- **TODO:** Thank thesis advisor, committee members, and readers who helped shape the research question and evaluation design.
- **TODO:** Acknowledge collaborators who contributed to the agentic workflow, MCP tooling, Ghidra integration, or testing infrastructure.
- **TODO:** Add any funding, lab, institute, or travel support acknowledgements required by the program.
- **TODO:** Add a brief personal acknowledgement paragraph once the final draft is complete.

Abstract

This thesis investigates whether tool-orchestrated agentic workflows can improve reverse engineering and malware analysis outcomes relative to weaker baselines. The system under study does not rely on a single monolithic agent. Instead, it coordinates planners, specialist workers, validators, and reporters through MCP-style tool access to Ghidra, static-analysis utilities, dynamic-analysis infrastructure, and supporting evidence-recovery tools. The thesis focuses on how staged reasoning, explicit tool routing, and evidence review affect tasks such as behavior reconstruction, configuration decoding, packing analysis, and malware-oriented triage.

This draft abstract is intentionally conservative. It establishes the thesis scope and argument structure without claiming finished results. **TODO:** Revise the abstract after the architecture, evaluation, and final quantitative findings are stable. **TODO:** State the primary research questions and final contribution claims in two or three sentences.

Table of Contents

Acknowledgements **iv**

Abstract **v**

1 Introduction **1**

1.1 Problem Motivation 2

1.2 Research Questions 2

1.3 Thesis Scope and Expected Contributions 3

1.4 Thesis Organization 3

2 Background and Problem Setting **5**

2.1 Reverse Engineering as an Evidence-Gathering Problem 5

2.2 LLM Agents, Tool Use, and Orchestration 6

2.3 MCP-Style Tool Integration 6

2.4 Binary Analysis Tooling in Scope 7

2.5 Problem Formulation for the Thesis 8

3 Related Work **9**

3.1 AI-Assisted Reverse Engineering and Malware Analysis 9

3.2 Tool-Using Agents and Multi-Agent Orchestration 10

3.3 Program Analysis Toolchains and Human Analyst Workflows 10

3.4 Evaluation of Security and Reverse-Engineering Systems 11

3.5 Thesis Positioning 11

4 System Overview **13**

4.1 End-to-End Workflow 13

4.2 Planner, Worker, Validator, and Reporter Roles 14
4.3 Configuration Surfaces 15
4.4 Shared State and Evidence Flow 15
4.5 Why This Architecture Fits Reverse Engineering 16

5 Implementation and Tool Integration 17

5.1 Repository Structure and Execution Path 17
5.2 Workflow Configuration 18
5.3 Ghidra Integration 19
5.4 Supporting Static and Dynamic Tooling 19
5.5 Artifacts, Bundles, and Provenance 20
5.6 User-Facing Monitoring and Result Inspection 21

6 Evaluation Methodology and Experimental Design 22

6.1 Evaluation Goals 22
6.2 Benchmark Corpus and Task Model 23
6.3 Single Runs, Sweeps, and Repetitions 24
6.4 Broad Coverage and Final Held-Out Malware Evaluation 26
6.5 Baselines and Experimental Variables 28
6.6 Judging and Scoring 29
6.7 Statistical Treatment and Comparison Strategy 30

7 Results and Analysis 32

7.1 Preliminary Repository Snapshot 32
7.2 Preliminary Development Observations 33
7.3 Broad Coverage Pilot Testing 35
7.4 Focused Decoder Development Study 37
7.5 Final Held-Out Evaluation Run 41
7.6 Qualitative Failure Analysis 43
7.7 Interpretation and Claim Boundaries 44

8 Limitations and Threats to Validity 45

8.1 Benchmark and Corpus Limitations 45
8.2 Model, Tool, and Infrastructure Variability 46
8.3 Judging and Measurement Limitations 46
8.4 Scope of Automation Claims 47
8.5 Ethical and Operational Considerations 47

9 Conclusion and Future Work **48**
9.1 Conclusion 48
9.2 Future Work 49

Bibliography **50**

Appendix A Supplementary Evaluation Material **51**
A.1 Sample and Task Inventory 51
A.2 Judge and Prompt Artifacts 51
A.3 Configuration and Experiment Provenance 52
A.4 Additional Figures or Tables 52

1

Introduction

Reverse engineering and malware analysis often require analysts to combine decompilation, string recovery, capability triage, configuration extraction, and selective dynamic investigation. That workflow is tool-rich and evidence-driven, but it is also time-consuming and difficult to automate well. This thesis studies whether an agentic system that explicitly orchestrates multiple tools can produce better binary understanding than weaker baselines that rely on simpler prompting or less structured analysis flows.

The project in this repository is centered on MCP-style tool orchestration rather than a single monolithic agent. The system under study combines a planner-worker-validator-reporter workflow with integrations for Ghidra, static-analysis utilities, dynamic-analysis or VM-assisted tooling, and supporting artifact-recovery capabilities. The thesis should therefore be framed around both systems questions and empirical questions: how the workflow is designed, why those design choices matter for reverse engineering, and whether they measurably improve task performance.

1.1 Problem Motivation

- Reverse engineering tasks are heterogeneous: some require structural reasoning over control flow, while others depend on recovered strings, decoded configuration blobs, packer handling, or behavioral corroboration.
- Single-shot prompting is poorly matched to this setting because useful evidence is distributed across many tools and intermediate artifacts.
- Tool-augmented agents may improve coverage and evidence quality, but they also introduce orchestration complexity, failure modes, cost, and reproducibility concerns.
- **TODO:** Write the motivating scenario that ties together malware triage, analyst workload, and the need for explicit evidence gathering.
- **TODO:** Explain why reverse engineering is a better stress test for tool-using agents than general coding or question-answering benchmarks.
- **TODO:** Clarify whether the thesis emphasizes malware analysis specifically, reverse engineering more broadly, or malware analysis as a motivating subdomain.

1.2 Research Questions

1. Does a tool-orchestrated agentic workflow improve binary-analysis task performance relative to weaker baselines?
2. Which parts of the workflow matter most: query shaping, worker specialization, validator topology, or tool availability?
3. How much of the improvement comes from access to external tooling versus prompt or coordination changes alone?

4. What failure modes remain even when the workflow is tool-augmented?

- **TODO:** Refine these into the final thesis research questions and explicitly map each one to a chapter or experiment.
- **TODO:** State the primary hypothesis and any secondary ablation hypotheses in advisor-approved wording.

1.3 Thesis Scope and Expected Contributions

- A modular agentic reverse-engineering system built around planners, workers, validators, and reporters.
- MCP-mediated integration of Ghidra, supporting static-analysis tools, and optional dynamic-analysis infrastructure.
- A binary-analysis evaluation harness built around sample-task pairs, baseline-first sweeps, repeated runs, and judge-driven scoring.
- A focused empirical investigation of whether tool-augmented agents outperform weaker baselines on reverse-engineering and malware-understanding tasks.
- **TODO:** Convert this outline into a final contribution list once the exact claims and chapter boundaries are fixed.
- **TODO:** Be explicit about what is not claimed, especially around full malware automation or replacing human analysts.

1.4 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 introduces the reverse-engineering problem setting and the technical background needed for the thesis.

Chapter 3 positions the work against prior literature on AI-assisted reverse engineering, malware analysis automation, and tool-using agents. Chapter 4 describes the system at a high level, and Chapter 5 details the implementation and tool integrations. Chapter 6 defines the experimental methodology. Chapter 7 is reserved for quantitative and qualitative findings. Chapter 8 discusses limitations and threats to validity, and Chapter 9 concludes with future work.

2

Background and Problem Setting

2.1 Reverse Engineering as an Evidence-Gathering Problem

Reverse engineering is not a single question answered by a single tool. Analysts move among disassembly, decompilation, string inspection, structural triage, pattern matching, and selective execution in order to construct an evidence-backed explanation of program behavior. This makes the domain a strong candidate for studying tool-orchestrated agents.

- Reverse engineering tasks often require both broad survey steps and narrow recovery steps.
- Malware-oriented questions raise the additional need for analyst caution, provenance tracking, and explicit uncertainty handling.
- The quality of an answer depends not only on the final narrative but also on whether the system gathered the right supporting artifacts.
- **TODO:** Define the task categories used in this thesis, such as default analysis, configuration recovery, behavior recovery, packing triage, and anti-analysis

investigation.

- **TODO:** State what counts as acceptable evidence for binary understanding claims in this thesis.

2.2 LLM Agents, Tool Use, and Orchestration

The system studied here is explicitly modular. Rather than expecting one model invocation to reason, search, inspect artifacts, and self-critique all at once, the workflow decomposes the task into stages with different responsibilities. This chapter should explain why that decomposition is appropriate for reverse engineering.

- Planner stages decompose the query and allocate work.
- Worker stages gather concrete evidence through tool use.
- Validator stages review coverage, evidence quality, and missing artifacts.
- Reporter stages synthesize the final answer for the run artifact.
- **TODO:** Explain the intuition behind planner-worker-validator-reporter style decomposition.
- **TODO:** Clarify how this differs from chain-of-thought prompting or single-agent tool use.
- **TODO:** Discuss why explicit stage contracts matter when tools can produce noisy or partial evidence.

2.3 MCP-Style Tool Integration

The repository uses MCP-style servers to expose tool capabilities to the runtime. This matters because the thesis argument is partly about interface design: how tool

affordances are surfaced to agents, how outputs are normalized, and how evidence is routed back into shared state.

- Core static lane: Ghidra, strings, FLOSS, capability analysis, hash lookups, YARA, binwalk, and related enrichers.
- Optional dynamic or execution-oriented lane: VM and sandbox-style tooling, packet or process observation, and task-specific execution probes.
- Artifact management layer: bundles, manifests, run records, judge outputs, and experiment aggregates.
- **TODO:** Document how MCP servers are loaded and partitioned into static versus dynamic tool groups.
- **TODO:** Explain the tradeoff between uniform tool access and tool-specific prompting or guardrails.
- **TODO:** Add a diagram showing how tool requests and results move through shared state.

2.4 Binary Analysis Tooling in Scope

This thesis should also introduce the technical role of each major tool class so later architecture chapters can assume that context.

- Ghidra provides decompilation, disassembly, symbol context, and structural inspection.
- String and FLOSS-style tooling surface literal and decoded text evidence.
- Capability and signature tools provide fast high-level hints that can be corroborated later.

- Dynamic or VM-backed tooling is reserved for cases where static evidence is insufficient or ambiguous.
- **TODO:** Explain why Ghidra is the primary structural analysis backend in this system.
- **TODO:** Clarify which tasks can be completed from static evidence alone and which ones benefit from dynamic execution.
- **TODO:** Describe how UPX, packing, and unpacking are treated as preprocessing versus analysis-time decisions.

2.5 Problem Formulation for the Thesis

At a high level, each experiment asks whether a workflow configuration can answer a binary-analysis task better than a baseline. In the maintained harness, the atomic evaluation unit is a sample-task pair. That framing should become the formal problem statement for the evaluation chapter.

- Inputs: binary sample, task prompt, tool availability, workflow configuration, and model configuration.
- Outputs: final analysis report, intermediate evidence artifacts, judge score, and run metadata.
- Comparison axis: score deltas, task success rates, error rates, and qualitative failure analysis across matched tasks.
- **TODO:** Convert this into a concise formal problem statement that the rest of the thesis can reference.
- **TODO:** State what a "weaker baseline" means in the context of this system and why that baseline is fair.

3

Related Work

This chapter should position the thesis against three overlapping research areas: AI-assisted reverse engineering and malware analysis, autonomous or semi-autonomous software agents, and binary-analysis evaluation methodology. The goal is not just to collect citations but to explain what technical gap this thesis fills.

3.1 AI-Assisted Reverse Engineering and Malware Analysis

- Prior work on language models for code understanding and security analysis.
- Prior work on automated malware triage, family classification, unpacking, capability extraction, or analyst-assistance workflows.
- Prior work on reverse engineering assistance that relies on decompiler output, symbolic artifacts, or prompt engineering.
- **TODO:** Survey literature on LLM-supported binary analysis and identify how much tool access those systems expose.

- **TODO:** Separate work that performs classification from work that performs open-ended reasoning about behavior, configuration, or provenance.
- **TODO:** Explain why malware triage and reverse engineering require more than summarizing decompiler text.

3.2 Tool-Using Agents and Multi-Agent Orchestration

- Research on agents that select tools, manage intermediate state, and decompose tasks.
- Planner-worker or manager-executor style systems that distribute work among specialists.
- Work on verification, critique, or self-review stages that resemble validator and reporter roles.
- **TODO:** Add citations for agent architectures that explicitly separate planning, execution, and review.
- **TODO:** Discuss what this thesis contributes beyond generic multi-agent orchestration, especially in binary-analysis constraints.
- **TODO:** Explain why explicit tooling contracts matter more here than in text-only benchmarks.

3.3 Program Analysis Toolchains and Human Analyst Workflows

- Literature on decompilers and binary-analysis frameworks such as Ghidra.
- Work on analyst workflows that combine strings, signatures, configuration extraction, and behavioral context.

- Research on when dynamic analysis is necessary to validate static conclusions.
- **TODO:** Cite foundational binary-analysis and reverse-engineering tooling papers or practitioner references.
- **TODO:** Describe where this thesis follows established analyst workflow and where it departs by automating orchestration.

3.4 Evaluation of Security and Reverse-Engineering Systems

- Benchmarks for malware or binary understanding.
- Human evaluation versus rubric-based judging.
- Repetition, paired comparisons, and significance testing in tool-augmented agent evaluations.
- **TODO:** Add citations for benchmark design and judging methodology in security-oriented AI systems.
- **TODO:** Explain why the thesis uses sample-task pairs, repeated runs, and judge-scored artifacts rather than only pass/fail labels.
- **TODO:** Position the custom experimental corpus relative to prior benchmarks and explain the tradeoffs.

3.5 Thesis Positioning

The distinctive claim of this thesis is not merely that language models can help with reverse engineering. The system contribution is that explicit tool orchestration, staged specialization, and evidence review can be made first-class design elements in a reverse-engineering workflow, and that those choices can be evaluated systematically rather than anecdotally.

- **TODO:** End the chapter with a concise statement of the gap in prior work that this thesis addresses.
- **TODO:** State why MCP-style tool orchestration is the right abstraction to highlight in the thesis narrative.

4

System Overview

This chapter should give the reader a coherent picture of the system before the implementation details appear. The emphasis should be on workflow, component roles, and evidence flow rather than low-level code.

4.1 End-to-End Workflow

At a high level, a run begins with a user query or evaluation task, resolves a workflow configuration, provisions tool access, and then executes a staged agent pipeline. The main system flow in this repository follows the path `main.py` → `config.py` → `frontend.py` → `runtime.py` → `pipeline.py`. The thesis should describe that flow conceptually rather than line by line.

- Query or task selection.
- Pipeline and architecture selection.
- Tool partitioning and runtime assembly.
- Planner, worker, validator, and reporter stage execution.

- Artifact capture, judging, and experiment aggregation.
- **TODO:** Add a figure for the run dataflow from input query to final report and judge artifact.
- **TODO:** Explain which parts of the workflow are static configuration choices and which parts are dynamic at run time.

4.2 Planner, Worker, Validator, and Reporter Roles

The workflow supports multiple stage kinds, but the core thesis story should focus on the planner-worker-validator-reporter topology.

- Planners decide how to decompose the task and which evidence should be gathered.
- Workers perform focused analysis passes, often with specialized tool access or role prompts.
- Validators review coverage, evidence quality, and missing artifacts before final reporting.
- Reporters convert intermediate evidence into the final narrative artifact used by the harness and judge.
- **TODO:** Describe the contract for each stage and the failure modes it is meant to catch.
- **TODO:** Explain why validators are treated as a first-class component rather than a final self-check by the reporter.
- **TODO:** Clarify how architecture presets change the available worker topology.

4.3 Configuration Surfaces

The maintained system exposes several configuration families that are important to the evaluation story:

- query variants
- worker subagent profiles
- worker prompt shape and role-prompt mode
- tool profiles
- architecture presets
- pipeline presets
- validator topology and validator review level

These are not merely engineering options. They are the intervention points used by the evaluation harness to compare baseline and variant behavior.

- **TODO:** Add a table mapping each configuration family to the aspect of behavior it is intended to test.
- **TODO:** Explain which options alter reasoning behavior, which alter available evidence, and which alter review strictness.

4.4 Shared State and Evidence Flow

The system keeps shared run state so later stages can inspect earlier findings, tool outputs, and failure metadata. This matters because reverse engineering benefits from explicit provenance. The thesis should explain what information is stored, how it is surfaced to later stages, and how that affects final reporting and judging.

- Intermediate tool outputs and structured findings.
- Stage status logs and work-item tracking.
- Final analysis report and supporting artifacts.
- Run metadata used by the evaluation harness, including bundle provenance and comparison context.
- **TODO:** Describe how shared state prevents evidence loss between planning, execution, validation, and reporting.
- **TODO:** Explain how the live monitor and result browser reflect this shared-state model.

4.5 Why This Architecture Fits Reverse Engineering

The value proposition of the architecture should be made explicit: binary analysis benefits from staged specialization because the evidence sources are heterogeneous, and high-confidence answers require both discovery and critique. A monolithic prompt can mention these needs, but it cannot enforce them structurally. This chapter should set up that argument so the evaluation chapter can test it.

- **TODO:** State the design principles that guided the architecture, such as modularity, traceability, evidence-first reasoning, and controllable ablations.
- **TODO:** Add a short example walkthrough of one analysis task moving through planner, workers, validator, and reporter.

5

Implementation and Tool Integration

This chapter should translate the high-level workflow into concrete implementation choices. It is the right place to explain which repository components implement the system, how tools are wired into the runtime, how artifacts are stored, and how analysis targets move through the harness.

5.1 Repository Structure and Execution Path

The implementation is split across the runtime package and the testing harness. The core workflow lives under `multi_agent_wf/`, while reproducible evaluation infrastructure lives under `Testing/`. This separation is itself important to the thesis because it distinguishes the research system from the measurement apparatus used to study it.

- `multi_agent_wf/config.py`: workflow configuration loading and normalization.
- `multi_agent_wf/runtime.py`: tool loading, worker topology assembly, and run-time orchestration.

- `multi_agent_wf/pipeline.py`: staged execution logic and stage contracts.
- `multi_agent_wf/shared_state.py`: intermediate artifact tracking and run-state mutation.
- `Testing/harness/`: bundles, judging, sweeps, results aggregation, recovery, and viewers.
- **TODO:** Describe why the runtime and evaluation harness are separated and what this buys for reproducibility.
- **TODO:** Add a diagram that maps code modules to conceptual system responsibilities.

5.2 Workflow Configuration

The workflow uses JSON-backed configuration for stage contracts, worker archetypes, architectures, prompts, and pipeline selection. That design is deliberate. It makes ablations and reproducible experiments easier.

- Pipeline presets determine which stages run and in what order.
- Architecture presets determine the available worker mix.
- Worker prompt shape, role-prompt mode, and review level tune behavior without rewriting the core runtime.
- **TODO:** Explain how configuration is validated and loaded into Python structures.
- **TODO:** Discuss why JSON workflow configuration is preferable to hard-coded stage logic for this research setting.

5.3 Ghidra Integration

Ghidra is the primary structural-analysis backend in the system. The thesis should describe not only that it is used, but how it is used: prepared bundles, exported analysis artifacts, and agent-facing access through tool mediation rather than ad hoc manual interaction.

- Ghidra-backed analysis is prepared through bundle generation.
- Agent stages consume Ghidra-derived artifacts. Where allowed, they can also query Ghidra-facing tools.
- The system explicitly tracks whether analysis is occurring on the original sample or on a derived artifact such as an unpacked UPX binary.
- **TODO:** Document the Ghidra bridge architecture and what metadata is exported into bundles.
- **TODO:** Explain how pending Ghidra changes or mutation requests are controlled.
- **TODO:** State which Ghidra capabilities are exposed to agents and which remain restricted.

5.4 Supporting Static and Dynamic Tooling

The broader tool ecosystem matters because much of the thesis is about tool augmentation rather than raw model capability. The implementation chapter should make the tool landscape explicit.

- Static support tools include strings, FLOSS, and hash lookups.

- Additional enrichers include capability extraction, YARA, binwalk, and UPX-oriented support.
- Dynamic support infrastructure includes VM-oriented tooling and execution-time instrumentation where the task demands it.
- Tool profiles are used to measure how much accuracy depends on the breadth of the tool ecosystem.
- **TODO:** Describe how tool servers are grouped into static and dynamic partitions.
- **TODO:** Explain how tool failures are surfaced back into run state and later analysis.
- **TODO:** Clarify which tasks in the benchmark corpus actually require dynamic evidence.

5.5 Artifacts, Bundles, and Provenance

Prepared bundles and run artifacts are central to the system's reproducibility story. Bundles capture analysis inputs and precomputed artifacts for a sample, while run records capture the outputs of one particular workflow configuration. The thesis should describe both layers.

- Bundles are reused when their generation inputs remain fresh.
- Prompt or persona changes do not by themselves invalidate bundles.
- Run records preserve final reports, judge outputs, live status, and per-task metadata.

- Recovery and repaired-experiment workflows preserve provenance instead of mutating original results in place.
- **TODO:** Explain the bundle lifecycle and why it is important for experiment cost and reproducibility.
- **TODO:** Describe how recovery runs and rebuilt experiments preserve provenance.
- **TODO:** Add one worked example of the files produced for a single sample-task run.

5.6 User-Facing Monitoring and Result Inspection

The project also includes monitoring and browsing interfaces that expose live state and archived results. These interfaces are worth mentioning because they operationalize the system's provenance model and make failure analysis tractable.

- Live monitor for in-progress experiments.
- Results browser for archived experiments, per-executable views, and detail panes.
- Recovery tooling for failed-task reruns and rebuilt repaired experiments.
- **TODO:** Briefly justify why these interfaces matter to the thesis instead of treating them as incidental engineering.
- **TODO:** Show how run, task, and comparison state become inspectable by the experimenter.

6

Evaluation Methodology and Experimental Design

This chapter should explain how the thesis evaluates the system without overstating what the current data can support. The repository already contains a concrete binary-analysis harness, and the final chapter should align tightly with that implementation.

6.1 Evaluation Goals

- Measure whether tool-orchestrated workflows outperform weaker baselines on binary-analysis tasks.
- Separate the effects of tool access, workflow topology, prompt shaping, and review strictness.
- Capture both successful analyses and structured failure modes such as analysis errors, validator blocks, and judge failures.
- **TODO:** State the primary evaluation objective in thesis-ready language.

- **TODO:** List the secondary ablation questions and map each one to a configuration family.

6.2 Benchmark Corpus and Task Model

The maintained binary benchmark uses an experimental corpus of authored executables and executable variants. The evaluation unit is a sample-task pair rather than only a sample. This detail is important because it lets the thesis distinguish broad understanding tasks from focused recovery tasks.

- Sample variants cover conditions such as stripping, packing, deceptive strings, anti-analysis behavior, and configuration decoding.
- Tasks include default analysis as well as focused recovery questions such as config extraction, decode-flow recovery, packing triage, and behavioral reconstruction.
- The corpus supports both breadth-oriented sweeps and depth-oriented studies on selected executables such as `config_decoder_test`.
- **TODO:** Describe the experimental corpus, including how many source programs, executable variants, and sample-task pairs were used for the final thesis experiments.
- **TODO:** Add a short table summarizing representative samples and the reasoning skills they are intended to stress.
- **TODO:** Explain why `config_decoder_test` became a centerpiece for deeper analysis.
- **TODO:** Explain how the final held-out malware-facing evaluation differs from the authored benchmark and why the two should not be conflated.

6.3 Single Runs, Sweeps, and Repetitions

The harness supports both single-run evaluation and experiment sweeps. Maintained sweeps are baseline-first and primarily one-variable-at-a-time. The thesis should explain this choice clearly, because it affects how causal claims can be interpreted.

- A global baseline defines the default workflow configuration.
- Variant families change one configuration surface at a time.
- Some families insert local family baselines where a fair comparison requires it.
- Repetitions support stronger claims about stability and significance.
- The current repository artifacts include a repaired breadth sweep with one repetition per configuration and a decoder-depth follow-up with three repetitions per configuration.
- **TODO:** Explain why the broad sweep uses one-variable-at-a-time comparisons instead of a full factorial design.
- **TODO:** Document the final repetition counts used for the headline experiments and for the decoder depth study. The current artifact set is asymmetric: the repaired broad sweep is descriptive because each configuration only has one repetition, whereas the decoder-depth combined experiment has three repetitions per configuration.
- **TODO:** Clarify how repaired experiments are handled when failed tasks are rerun and merged.
- **TODO:** Make explicit that repaired and merged experiments preserve raw run provenance, rather than replacing the original experiments in-place.

For reproducibility, the maintained repository entry points for these development experiments are now the folder-based launch presets under `Testing/config/presets/`. The broad-coverage pilot discussed in this thesis corresponds to the preset `coverage_broad_r1_60usd`. The repeated decoder-depth case study corresponds to `sweep_decoder_depth_r3`, and the narrower follow-up depth suite corresponds to `sweep_decoder_depth_followups_r3`. A clean rerun begins by rebuilding and preparing the experimental corpus:

```
make all-exes
python Testing/scripts/prepare_bundles.py --corpus experimental
```

The maintained preflight and execution commands for the broad pilot are:

```
python Testing/scripts/run_launch_preset.py \
  --preset coverage_broad_r1_60usd \
  --judge-model openai:gpt-4o-mini \
  --preflight-only

python Testing/scripts/run_launch_preset.py \
  --preset coverage_broad_r1_60usd \
  --judge-model openai:gpt-4o-mini
```

The maintained preflight and execution commands for the repeated decoder-depth suites are:

```
python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_r3 \
  --judge-model openai:gpt-4o-mini \
  --preflight-only
```

```
python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_r3 \
  --judge-model openai:gpt-4o-mini

python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_followups_r3 \
  --judge-model openai:gpt-4o-mini \
  --preflight-only

python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_followups_r3 \
  --judge-model openai:gpt-4o-mini
```

Preflight-only commands now write readiness artifacts under `Testing/results/preflight/`. Real launched experiments write their run catalogs, child runs, summaries, and figures under `Testing/results/experiments/<experiment\_id>/`. This separation is important because it keeps the experiment directory reserved for actual executed sweeps rather than planning scaffolds.

6.4 Broad Coverage and Final Held-Out Malware Evaluation

The methodology chapter should separate three evaluation roles:

- development and pilot runs on the authored benchmark, used to debug workflow behavior and identify promising interventions;
- broader malware-facing coverage checks, used as a realism test over manually supplied families; and

- the final held-out evaluation run, which should carry the main thesis-grade empirical claims.

The repository now includes a metadata-backed manual corpus scaffold for analyst-supplied malware families rather than a live malware corpus. This is important both for safety and for scientific clarity: the broad-coverage plan should be described as a broader realism check, while the final held-out run should be described as the main evaluation.

- The intended five-family set is Emotet, QakBot, TrickBot, WannaCry, and PlugX, but the repository stores only placeholder paths and evaluator-side metadata.
- The manual final-evaluation manifest uses neutral numbered runtime aliases, while family labels and gold-label pointers remain on the evaluator side.
- The current repository preset for this stage is a scaffold, not yet the completed final run. It exists to lock the intended evaluation structure before the analyst-supplied samples are inserted.
- The active final-run scaffold currently uses one blind default-analysis pass per sample, then spends the remaining budget on a wider comparison across worker breadth, prompt shape, tool surface, and architecture.
- **TODO:** Describe the manual final-evaluation manifest and explain why the repository stores metadata only, not binaries or fetch automation.
- **TODO:** Document the blind runtime alias policy and explain why family identity should not be exposed to the analysis agent during the run.
- **TODO:** Explain how broad-coverage pilot testing informs method development without being presented as proof of general effectiveness.

- **TODO:** State the criteria for promoting a configuration from benchmark-development runs into the final held-out malware evaluation.

6.5 Baselines and Experimental Variables

The current harness exposes several variable families that can be studied empirically:

- query verbosity
- worker subagent profile
- worker prompt shape and worker role-prompt mode
- tool availability
- architecture preset
- pipeline preset
- validator topology
- validator review level
- **TODO:** Decide which of these families will be discussed in the thesis body versus moved to appendix material.
- **TODO:** Explain the rationale for the chosen baseline in the final experiments.
- **TODO:** Document the deeper `config_decoder_test` study separately from the broad sweep. The current repaired broad sweep suggests that the no-role worker-prompt condition and the single-generalist worker profile are the most interesting follow-up families, which is why the depth study focuses on those surfaces plus a validator-heavy pipeline comparison.

- **TODO:** Justify why the brief-query condition was not promoted into the decoder-depth follow-up despite a positive descriptive broad-sweep delta; the current interpretation is that it looked less mechanistically interesting than worker-prompt and worker-subagent effects.

6.6 Judging and Scoring

Run outputs are judged with a structured binary-analysis rubric and converted into continuous scores. This thesis should make the judging methodology explicit, including both its usefulness and its limitations.

- Per-task judge outputs preserve structured feedback and an overall score.
- Legacy pass or fail thresholds still exist, but sweep analysis should focus on score deltas, success rates, and error rates.
- The harness preserves analysis outcomes such as completed, analysis error, no result, judge error, and validator blocked.
- The final held-out evaluation scaffold also includes analyst-reviewed gold-label templates. These are intended to anchor evidence expectations and false-positive controls, not merely family attribution.
- **TODO:** Describe the rubric dimensions used by the judge and justify why they align with reverse-engineering quality.
- **TODO:** Explain whether human spot-checking was used to validate judge outputs and how analyst-reviewed gold labels were incorporated in the final held-out run.
- **TODO:** Discuss the difference between completed-but-weak analyses and infrastructure failures.

- **TODO:** Document the final held-out weighting policy: evidence recovery, capability inference, reasoning quality, false-positive control, and optional low-weight family attribution.

6.7 Statistical Treatment and Comparison Strategy

The final thesis should not report only descriptive movement. The harness supports paired comparisons and limited significance analysis when repetitions or enough matched task pairs exist.

- Replicate-level comparison when repeated runs exist.
- Paired-task comparison across matched sample-task pairs.
- Reporting of score deltas, task success deltas, and significance outputs where supported.
- Explicit distinction between descriptive one-off sweep results and repeated experiments that can support stronger stability claims.
- **TODO:** State the statistical tests actually used in the final thesis and justify their assumptions. In particular, explain that the repaired broad sweep should be reported descriptively because it has one repetition per configuration, while the decoder-depth combined experiment can support limited repeated-trial summaries.
- **TODO:** Document how missing or recovered tasks are treated in the analysis set.
- **TODO:** Explain what level of evidence is sufficient for claims in the results chapter. The current artifact set supports stronger claims about the decoder-depth study than about the one-repetition broad sweep.

- **TODO:** Describe how validator-blocked outcomes are separated from infrastructure failures and why collapsing them into a single failure rate would be misleading.

7

Results and Analysis

This chapter should distinguish clearly between early-stage development evidence and the final thesis-grade evaluation. The repository already contains useful experimental artifacts, but the current materials are best treated as pilot or methodology-shaping results unless they are explicitly identified as part of the final held-out evaluation.

7.1 Preliminary Repository Snapshot

The repository currently contains two result sets that are useful for shaping the final analysis chapter:

- a repaired breadth-oriented sweep artifact, which compares several configuration families but only has one repetition per configuration; and
- a merged decoder-depth artifact, which compares five configurations on the `config_decoder_test` stripped and packed variants with three repetitions each.

These artifacts are useful because they expose failure modes, refine the reporting stack, and help identify which workflow variables deserve deeper study. They should

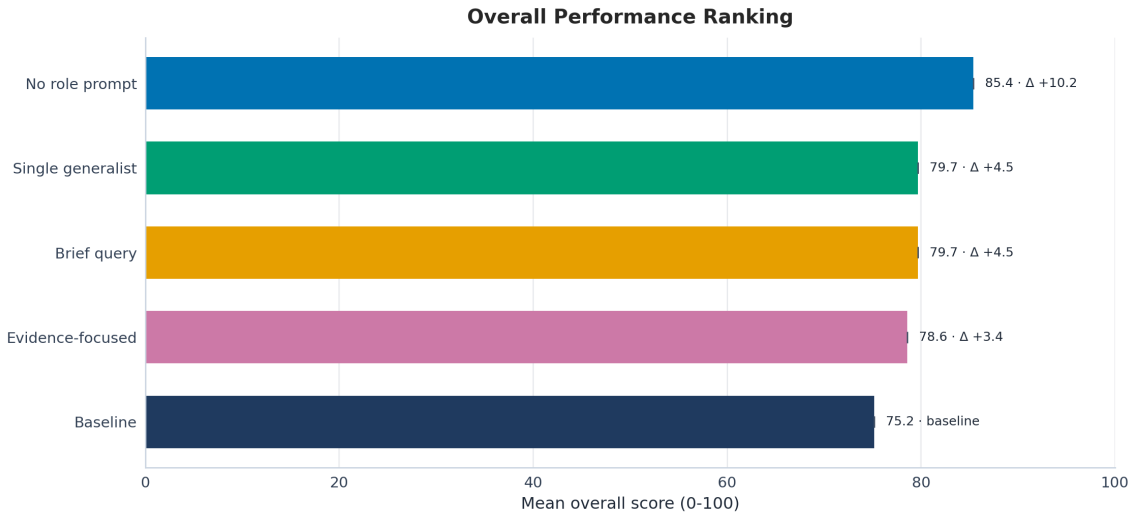

Figure 7.1: Preliminary repaired broad-sweep ranking. This figure is useful for hypothesis generation and follow-up planning rather than for a final claim about general effectiveness.

not, however, be written up as the final answer to the thesis research question without a separate held-out evaluation. In particular, the breadth sweep is descriptive rather than statistically strong because each configuration only appears once.

7.2 Preliminary Development Observations

- Early runs are most useful for debugging workflow behavior, understanding failure accounting, and selecting promising intervention families.
- At this stage, the goal is not to prove broad effectiveness but to identify which comparisons are worth carrying forward into stronger evaluation.
- All numeric observations in this section should be treated as provisional unless they are later repeated in the final evaluation section.

Provisional observations from the repaired broad sweep currently on disk:

- The no-role worker-prompt condition shows the largest descriptive score lift

Figure 7.2: Preliminary repaired broad-sweep runtime and cost tradeoffs. The main use of this figure is to identify follow-up candidates and obvious cost or latency regressions before the final held-out evaluation.

over baseline in this repaired pilot sweep: mean score `85.405` versus baseline `75.155`, with a score delta of `+10.250`. This should be treated as a follow-up signal rather than as definitive evidence.

- The single-generalist worker profile and the brief-query condition both improve mean score by roughly `+4.5`. The single-generalist condition is more useful for follow-up work because it changes the workflow structure rather than only shortening the query surface.
- After repair, the retained breadth-sweep rows have full coverage. That is important for debugging the harness, but it does not remove the limitation that the sweep still has one repetition per configuration.
- **TODO:** Keep this section explicitly framed as preliminary in the final thesis draft.
- **TODO:** Replace the provisional repository-snapshot discussion above with a short bridge into the final evaluation once that run exists.

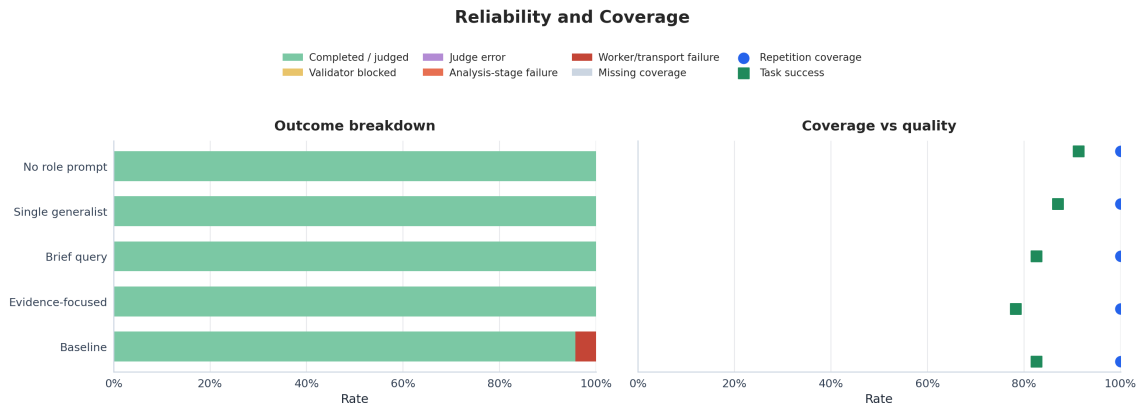

Figure 7.3: Preliminary repaired broad-sweep reliability and coverage view. Even in pilot runs, the reporting should keep validator, analysis, and infrastructure failures separate.

- **TODO:** Explain why the broad sweep was used to choose follow-up variables rather than as the sole basis for a strong final claim.
- **TODO:** Report cost, runtime, and stability tradeoffs alongside score movement so the reader sees why score alone was not treated as decisive at this stage.

7.3 Broad Coverage Pilot Testing

The repaired breadth sweep is best treated as a broad-coverage pilot over the authored benchmark. Its role in the thesis should be to show how the harness was used to explore the configuration space, surface failure modes, and choose promising follow-up conditions before any held-out malware-facing evaluation.

- This stage should be described as a broader realism check over task categories and workflow families, not as proof of general external validity.
- It is appropriate to use this stage to motivate which interventions were carried into the focused depth study and later held-out evaluation.

- It is not appropriate to treat this stage alone as evidence that the system generalizes to real malware families.
- If a manually supplied malware-family pilot is run before the final locked evaluation, it should also be framed here as pre-final and methodology-shaping rather than thesis-conclusive.
- **TODO:** Add a concise table that summarizes the retained pilot families, their descriptive score movement, and the reason each family was or was not advanced into later testing.
- **TODO:** Use this section to explain what the broad pilot taught about methodology, not just which variant happened to score highest.
- **TODO:** If a malware-family realism check is run before the final held-out evaluation, report it here as a broad-coverage pilot and state clearly that it is still pre-final.

The maintained reproduction path for this section is the preset `coverage_broad_r1_60usd`. A clean rerun uses `Testing/scripts/run\_launch\_preset.py` with a preflight pass first and then the real run command:

```
python Testing/scripts/run_launch_preset.py \
  --preset coverage_broad_r1_60usd \
  --judge-model openai:gpt-4o-mini \
  --preflight-only

python Testing/scripts/run_launch_preset.py \
  --preset coverage_broad_r1_60usd \
  --judge-model openai:gpt-4o-mini
```

The preflight record for this pilot is written under `Testing/results/preflight/`, while the executed sweep and its figures are written under `Testing/results/experiments/<experiment\_id>/`. That directory is the canonical source for the `variant\_summary.csv`, `task\_comparison.csv`, and `outputs/*.png` artifacts cited in this section.

7.4 Focused Decoder Development Study

The deeper `config_decoder_test` study should serve as a focused development case study rather than just another row in the broad sweep. Its purpose is to test whether the selected interventions appear to improve depth of understanding on a deliberately richer executable, not to replace the final held-out evaluation.

- Compare stripped versus packed-and-stripped variants.
- Compare default analysis with focused configuration-recovery tasks.
- Analyze whether the selected variable family improves depth of understanding or only broad summary quality.

Provisional observations from the current combined decoder-depth experiment:

- Baseline mean score is `79.232` with task-success rate `0.963`.
- The no-role worker-prompt condition appears to be the strongest repeated-trial variant in this focused study: mean score `83.597`, score delta `+4.365`, lower mean runtime, and nearly flat cost relative to baseline. This is promising, but it remains a case-study result rather than the final thesis endpoint.
- The single-generalist worker profile also improves mean score to `82.671`, but its task-success rate drops slightly from `0.963` to `0.926`. The current interpretation is therefore a tradeoff, not an unconditional improvement.

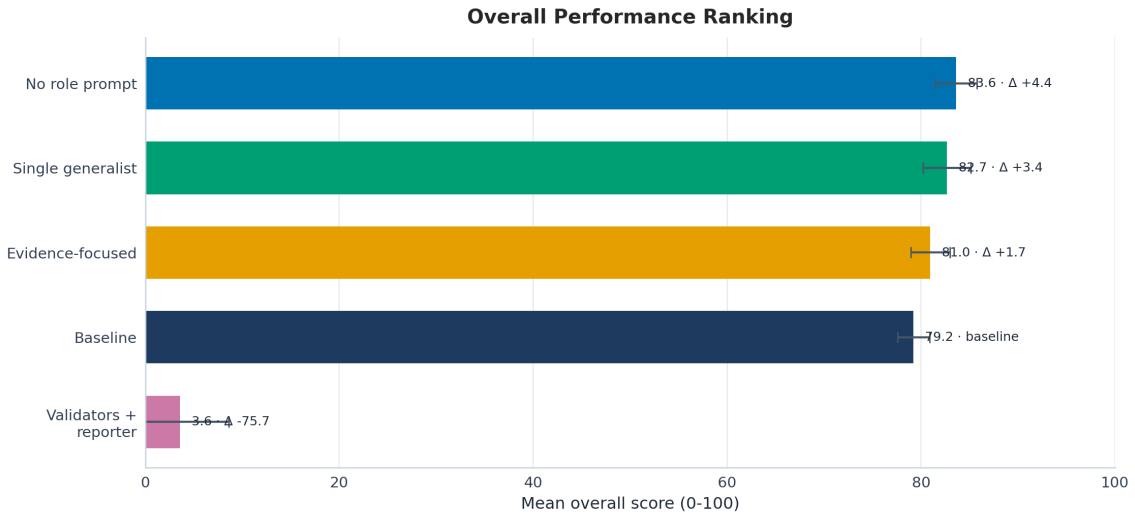

Figure 7.4: Preliminary decoder-depth ranking across stripped and packed `config_decoder_test` tasks. This figure is stronger than the broad pilot because it includes repetitions, but it is still part of the developmental evidence base rather than the final held-out evaluation.

- The validator-heavy pipeline appears to perform poorly in this study: mean score `3.571`, task-success rate `0.037`, validator-blocked rate `0.963`, and much larger runtime and cost than baseline. This is useful as an early warning about workflow design, not just as a low score.
- Task-level outputs suggest that the packed UPX variant remains methodologically informative: for example, the no-role worker-prompt condition improves the packed default-analysis task by `+10.713`, while the single-generalist worker profile improves the packing triage task but regresses the parser-validation audit task.
- **TODO:** Turn the working observations above into a polished preliminary case-study narrative once the wording is locked.
- **TODO:** Explain why `config_decoder_test` is a useful stress test for reverse-engineering depth.

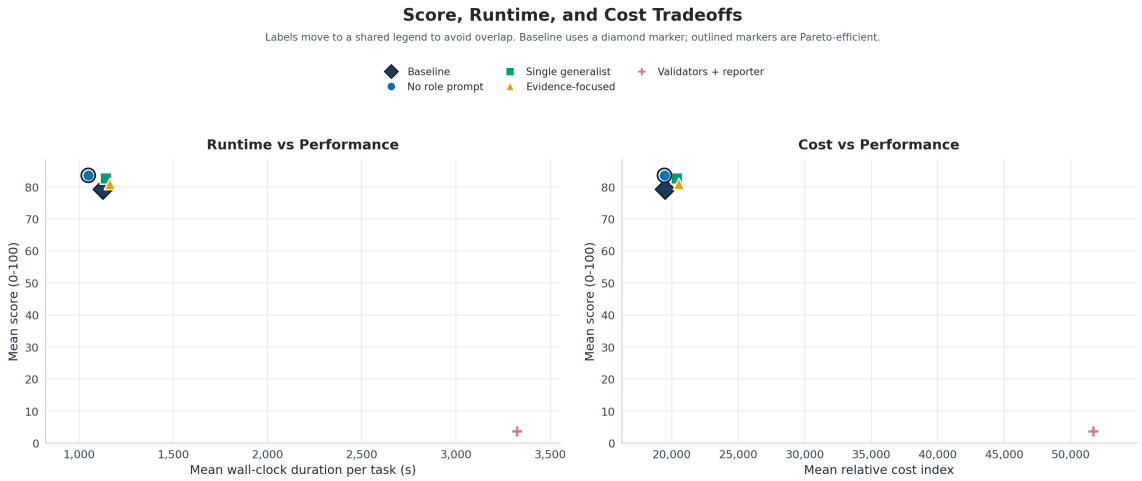

Figure 7.5: Preliminary decoder-depth tradeoffs. This view is useful for understanding whether a score gain comes with a practical runtime or cost penalty.

- **TODO:** Discuss whether the packed variant changes the relative value of tool orchestration or unpacking support. The current task-level evidence suggests that packed-sample handling changes which intervention helps: prompt-shape changes help some packed tasks, while worker-subagent structure helps others.
- **TODO:** Explain why the no-role worker-prompt condition looks better in the repeated decoder-depth study than the evidence-focused worker prompt, and whether that difference reflects cleaner worker instructions or something specific to this benchmark.
- **TODO:** Analyze the validator-heavy pipeline separately as a failure mode, not just a low-scoring configuration. Its near-total validator-block rate is methodologically interesting because it changes what "failure" means.

The maintained reproduction paths for the decoder-depth studies are the presets `sweep_decoder_depth_r3` and `sweep_decoder_depth_followups_r3`. The first preset reproduces the repeated combined depth comparison over the stripped and

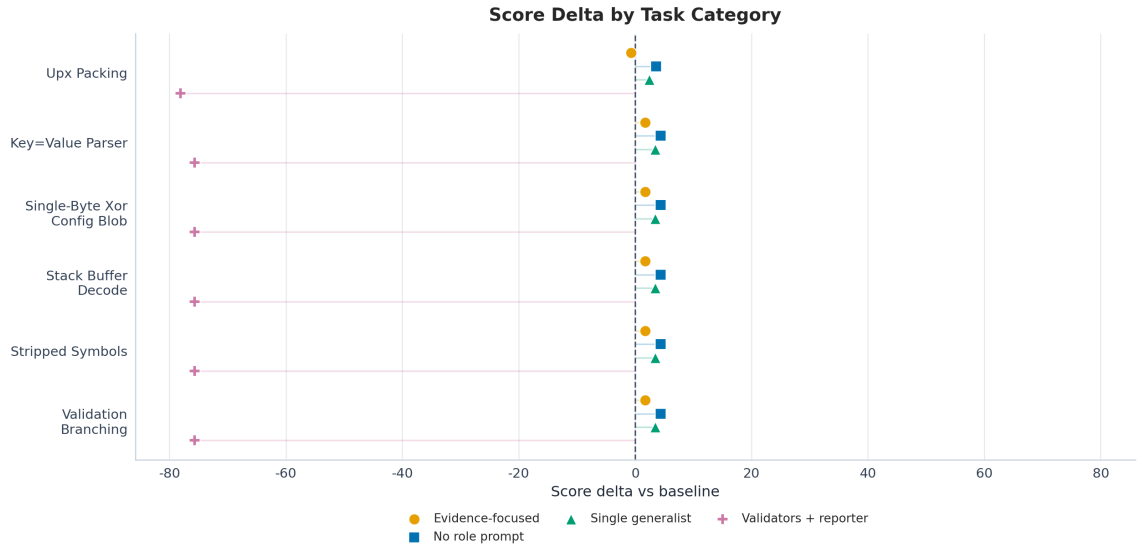

Figure 7.6: Preliminary decoder-depth task-category score deltas. This figure supports a case-study interpretation of where each intervention helps or hurts.

packed decoder variants. The second reproduces the narrower worker-prompt and validator follow-up suite. The repository commands are:

```
python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_r3 \
  --judge-model openai:gpt-4o-mini \
  --preflight-only

python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_r3 \
  --judge-model openai:gpt-4o-mini

python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_followups_r3 \
  --judge-model openai:gpt-4o-mini \
```

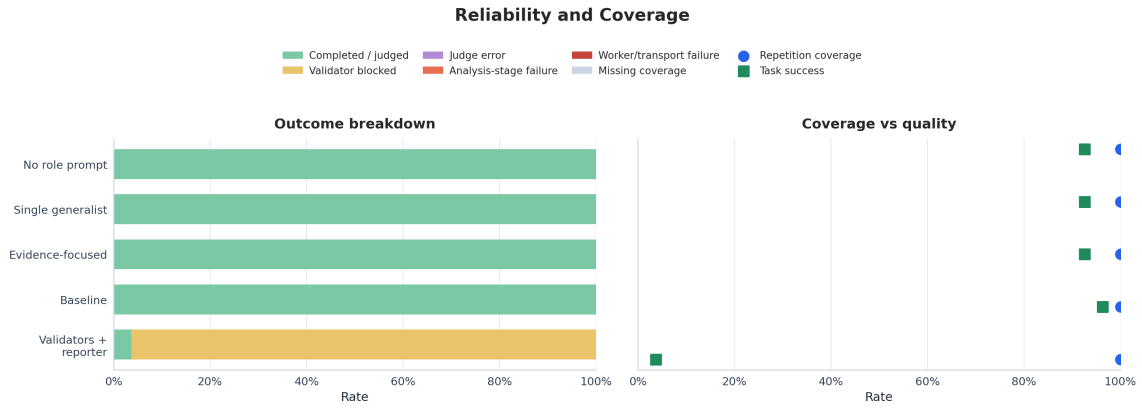

Figure 7.7: Preliminary decoder-depth reliability and coverage view. This figure is especially useful because it distinguishes a validator-dominated failure mode from simple infrastructure failure.

```
  --preflight-only
```

```
python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_followups_r3 \
  --judge-model openai:gpt-4o-mini
```

As with the broad pilot, preflight writes only to `Testing/results/preflight/`. Executed decoder-depth suites write their canonical outputs under `Testing/results/experiments/<experiment\_id>/`, where the thesis-facing comparison tables and figures can be regenerated directly from the run artifacts.

7.5 Final Held-Out Evaluation Run

This section should be reserved for the main thesis-grade evaluation. The final write-up should make it obvious that this is the section that carries the strongest empirical claims, while the earlier sections remain developmental or pilot evidence.

- The final held-out run should use the locked malware-family set, final workflow configuration choices, and final reporting language.

- Broad-coverage pilot testing and authored-benchmark development runs should be treated as preparation for this section, not as a substitute for it.
- Any stronger language about effectiveness, robustness, or comparative superiority should be grounded here rather than in the preliminary sections above.
- The current repository scaffold for this section is the blind manual-corpus preset `thesis_final_eval_r3_100usd`, which compares the baseline against a wider but still selective set of development-informed follow-up conditions over five analyst-supplied samples, one locked blind default-analysis task per sample, and three repetitions.
- Runtime analysis should remain blind to malware family identity. The evaluator-side manifest stores family and provenance metadata, but the runtime should only see neutral blind aliases plus task prompts that treat the binary as unknown.
- **TODO:** Document the final malware-family set: Emotet, QakBot, TrickBot, WannaCry, and PlugX, using analyst-supplied samples only.
- **TODO:** State the final run configuration(s), including baseline, treatment conditions, repetition count, the use of blind runtime aliases, and any held-constant tool or model settings.
- **TODO:** Insert the final evaluation tables and figures for score, runtime, cost, reliability, and coverage.
- **TODO:** Compare the final evaluation configurations against the chosen baselines using the final scoring and failure-accounting policy.

- **TODO:** Describe the analyst-reviewed gold-label workflow, including where Ghidra/manual evidence anchors were inserted and how unsupported claims were encoded.
- **TODO:** Add the final statistical analysis and confidence language here, including any caveats about sample size, power, multiple comparisons, and budget-driven retry variance.
- **TODO:** Make explicit which claims are supported by the final held-out run versus only by preliminary development evidence.

7.6 Qualitative Failure Analysis

The thesis should include concrete examples of failures, not just averages. This is particularly important in agentic systems where some runs fail because the workflow reasoned poorly, while others fail because the tool chain or orchestration broke down.

- Analysis errors and worker assignment failures.
- Cases where the report completed but important evidence was missing.
- Cases where recovery reruns changed the aggregate interpretation.
- **TODO:** Select two or three illustrative failure cases and analyze them in detail.
- **TODO:** Differentiate reasoning failures, tool failures, and judging or recovery artifacts. A good starting point is to contrast the validator-blocked `pipeline_preset:validators_reporter` runs with repaired broad-sweep failures that were caused by worker assignment or analysis-stage errors.
- **TODO:** Show how the live monitor or result browser made these failures inspectable.

- **TODO:** Include at least one example where recovery changed the aggregate interpretation, and one example where recovery was intentionally not used because the underlying failure was deterministic rather than transient.

7.7 Interpretation and Claim Boundaries

- Which interventions improve evidence quality?
- Which interventions merely change style or verbosity?
- Which workflow changes improve some task types while hurting others?
- What do the results imply for future reverse-engineering assistants?
- Which observations are preliminary and which are supported by the final held-out evaluation?
- **TODO:** End the chapter with a concise synthesis of what the final evaluation implies for the thesis research questions.
- **TODO:** Ensure every claim here is supported by a table, figure, or explicitly cited run artifact. Broad pilot claims should remain explicitly descriptive, and stronger language should be reserved for the final held-out evaluation section.
- **TODO:** Make explicit that family attribution, if reported at all, is low weight compared with evidence recovery and false-positive control.

8

Limitations and Threats to Validity

This thesis should be explicit about what the system and experiments do not establish. The project already has enough complexity that a serious limitations chapter will strengthen the final argument.

8.1 Benchmark and Corpus Limitations

- The benchmark corpus is authored and curated rather than harvested from unrestricted real-world malware collections.
- Sample-task pairs cover important reverse-engineering skills, but they cannot span the full diversity of malicious software.
- Some tasks are intentionally interpretable and may not capture the hardest real-world analyst workflows.
- **TODO:** State how representative the experimental corpus is and where it is intentionally simplified.

- **TODO:** Discuss any bias introduced by using authored samples whose ground truth is known.

8.2 Model, Tool, and Infrastructure Variability

- Model behavior may drift over time even under the same nominal configuration.
- Tool outputs can change with version upgrades or environment differences.
- Dynamic-analysis infrastructure adds host and VM variability that can affect reproducibility.
- **TODO:** Document which versions, APIs, and runtime environments were fixed for the final experiments.
- **TODO:** Explain how bundle reuse and artifact tracking mitigate, but do not eliminate, this problem.

8.3 Judging and Measurement Limitations

- Judge scores are a proxy for analyst usefulness, not the same thing as human expert review.
- Some tasks may reward well-structured reporting more than deep mechanistic understanding.
- Recovered runs and rebuilt experiments improve completeness but may complicate strict independence assumptions.
- **TODO:** Discuss how judge quality was validated and where the rubric may still be weak.
- **TODO:** Explain any remaining mismatch between automated scoring and analyst value.

8.4 Scope of Automation Claims

- The system assists reverse engineering; it does not eliminate the need for expert analyst review.
- Tool orchestration can improve evidence gathering without guaranteeing correctness of interpretation.
- Automatic unpacking or dynamic execution can help analysis but can also alter the artifact under study or introduce new risks.
- **TODO:** Be precise about what kinds of analyst effort this system reduces and what it still cannot replace.
- **TODO:** Address the correctness and security risks of automatically switching to unpacked artifacts.

8.5 Ethical and Operational Considerations

- Malware-oriented tooling creates dual-use concerns.
- Automated binary analysis can surface dangerous capabilities or operational details.
- Safer experimentation may require containment, logging, and restricted dynamic execution.
- **TODO:** Add a short ethics discussion covering dual-use risk, containment, and responsible publication choices.
- **TODO:** State any restrictions on sharing binaries, prompts, or unpacked artifacts in the final appendix or artifact release.

9

Conclusion and Future Work

This chapter should close the thesis by restating the problem, summarizing the main findings, and identifying the next research steps. It should remain tightly aligned with the actual evidence presented in Chapter 7.

9.1 Conclusion

- Restate the motivation for tool-orchestrated agentic reverse engineering.
- Summarize the system contribution.
- Summarize the main empirical takeaway without overstating certainty.
- Distinguish clearly between what was learned from preliminary development runs and what was established by the final held-out evaluation.
- **TODO:** Write a concise closing argument that answers the thesis research questions directly.
- **TODO:** State the strongest defensible claim supported by the final held-out experiments, not just by the earlier development or pilot runs.

- **TODO:** Add one sentence that explicitly limits the interpretation of the preliminary broad-coverage and decoder-depth runs to hypothesis-shaping and methodology refinement where appropriate.
- **TODO:** State clearly that the held-out malware run was designed to keep runtime family identity blind and to prioritize evidence recovery over family naming.

9.2 Future Work

- Broader and more realistic malware corpora.
- Stronger human evaluation and analyst-in-the-loop studies.
- Richer dynamic-analysis orchestration and sandbox integration.
- More principled automatic artifact selection, including packed versus unpacked analysis targets.
- Improved cost-aware planning, recovery, and significance analysis.
- **TODO:** Prioritize the future-work items that best follow from the actual limitations chapter.
- **TODO:** Call out which extensions are engineering scale-ups versus genuinely new research questions.

Bibliography

TODO: Add curated related-work, tooling, and methodology citations here once the literature review chapter is fully drafted. Re-enable BibTeX after real citations are inserted into the document.

Appendix A

Supplementary Evaluation Material

This appendix should collect material that is useful for reproducibility but too detailed for the main text.

A.1 Sample and Task Inventory

- **TODO:** Add a compact table of the executables used in the final thesis experiments.
- **TODO:** Summarize the task families attached to each sample, especially the deeper `config_decoder_test` study.
- **TODO:** Document which samples are stripped, packed, or otherwise designed to stress specific tooling.

A.2 Judge and Prompt Artifacts

- **TODO:** Include abbreviated judge rubric dimensions or a representative rubric excerpt.

- **TODO:** Include representative task prompts and note how query variants transform them.
- **TODO:** Add any prompt templates necessary to understand planner, worker, validator, and reporter stages.

A.3 Configuration and Experiment Provenance

- **TODO:** List the main workflow configuration files that define architectures, pipelines, and tool profiles.
- **TODO:** List the evaluation configuration files that define sweeps, presets, and corpora.
- **TODO:** Explain how repaired experiments and recovery sessions should be cited in the final artifact package.

A.4 Additional Figures or Tables

- **TODO:** Move large comparison tables, sample manifests, or result-browser screenshots here if they interrupt the narrative in the main text.
- **TODO:** Add supplementary error-analysis tables if they are too detailed for the main results chapter.